

Biosignal Processing

Lab I: Deterministic filtering and linear stochastic models with applications to ECG and EEG

Maria Alonso and Claudia Gallego

September 14, 2026

1 Introduction

The objective of this laboratory is to study signal processing techniques commonly used in biomedical analysis. It consists of a pre-lab task and three lab tasks: ECG signal enhancement, AR-model estimation and EEG-signal classification for brain-computer interface

2 Pre-Laboratory Preparation

2.1 Digital Filter Design: FIR vs. IIR

When designing a filter to process a signal we need to choose between mainly two different kinds of digital filters: FIR and IIR.

FIR filters are characterized by being able to achieve an exact linear phase response when its impulse response is either symmetric or antisymmetric. The linearity ensures that all frequency components of a signal are delayed by the exact same amount of time while preserving the time-domain morphology of the wave perfectly. However, the bad part of these filters is their computational cost. To achieve a steep transition band and a deep stopband attenuation it is required a very long impulse response (a high filter order), this results in a heavy computational load due to all the number of multiplications required per sample.

On the other hand, IIR filters can easily meet strict magnitude specifications with a significantly lower filter order. Since this kind of filters uses feedback, when designing you have the freedom to position both poles and zeros (unlike FIR filters that only uses zeros). This feature reduces the required number of multiplications making them really efficient. The trade off of using IIR filters is that they introduce signal distortion due to the nonlinear phase response. Also different frequencies are delayed by varying amounts of time, which alters the morphology of the output wave.

2.1.1 Mitigating IIR Phase Distortion

To solve the non-linearity of IIR filters it is used the forward-backward filtering technique. Where the input signal is first processed forward through the IIR filter results in an output that is reversed in time and is passed through the filter for a second time. Finally, the doubly filtered signal is reversed in time again to produce the final output. Passing the signal through the

filter in opposing directions cancels out the phase distortion, resulting in a zero-phase transfer function.

While powerful, this technique has limitations in practical applications. Primarily, the overall effective filter order is doubled, and it is strictly an offline processing technique due to the requirement for time-reversing the signal. While it can be approximated in "almost" real-time by processing overlapping signal segments, this introduces a systemic delay. Additionally, applying this technique at very high frequencies (e.g., above 1000 Hz) can cause the filter to become unstable as the poles are pushed closer to the unit circle. Finally, forward-backward filtering is difficult to adapt for time-variant filtering scenarios where the cutoff frequency must dynamically change over time.

2.2 Autoregressive Signal Modeling

When analyzing complex signals like EEGs, it is often used an Autoregressive (AR) model. This kind of model predicts the current value of a signal by looking at a specific number of its past values, plus some random noise.

2.2.1 Defining the AR(p) Process

An AR process of order p (this means that it looks at p past values) is defined by this equation:

$$x(n) + \sum_{k=1}^p a_k x(n-k) = e(n)$$

Where the following assumptions are made for its correct use:

1. **The Signal $x(n)$:** Must be “wide-sense stationary” (WSS) with a mean of zero. This means its basic statistical properties do not change over time.
2. **The Parameters a_k :** The filter coefficients must be constant and create a mathematically stable system.
3. **The Noise $e(n)$:** Must be stationary “white noise.” It must have a mean of zero, a constant variance (σ^2), and be completely uncorrelated with past signal values.

Since it is assumed that $e(n)$ is perfect white noise, its covariance function is:

$$r_e(k) = \sigma^2 \delta(k)$$

This formula states that the noise is unpredictable except at lag zero ($\delta(k)$ is 1 when $k = 0$, and 0 everywhere else). If the noise *was* predictable (if it had non-zero covariance at other lags), it would mean that the design model failed to capture some important, predictable information from the actual EEG signal.

2.2.2 Estimating Parameters with Yule-Walker Method

To make the AR model work, it is needed to find the actual values of the parameters (a_k). This is done using the Yule-Walker method.

Yule-Walker works by taking the AR equation and multiplying it by past signal values to relate the parameters directly to the signal's autocorrelation (how much the signal resembles its past self). This creates a system of linear equations, which simplifies to:

$$Ra = -r$$

This method is useful because it turns a complex modeling problem into matrix algebra. So, the structure of the R matrix guarantees that the designed filter is mathematically stable.

2.2.3 Choosing the Right Model Order

To decide how many past values (p) the model should look at, it is utilized criteria that penalize the addition of parameters since if the order is too high the error variance would drop but the model would start memorizing random noise instead of the actual signal's trend (overfitting). The criteria are the following:

- **Akaike Information Criterion (AIC):** $AIC(k) = N \ln(\hat{\sigma}_k^2) + 2k$
- **Minimum Description Length (MDL):** $MDL(k) = N \ln(\hat{\sigma}_k^2) + k \ln(N)$

These formulas are calculated for a range of orders and choose the one that gives the lowest score. It is generally preferred MDL for large datasets (N) because its penalty for complexity ($k \ln(N)$) is heavier than AIC's ($2k$). This ensures to select a simpler, more robust model that doesn't overfit the noise.

2.2.4 The Danger of Underfitting

If you pick a model order that is too low, mathematical bias is introduced. This can be proved by trying to fit a simple AR(1) model to a signal that was actually generated by a more complex AR(2) process.

If the true process is AR(2), the Yule-Walker equations show that the relationship between its covariance and parameters is:

$$\frac{r_x(1)}{r_x(0)} = \frac{-a_1}{1 + a_2}$$

If we forcefully restrict our model to AR(1), the Yule-Walker equation calculates our single parameter ($\tilde{a}_1$) as:

$$\tilde{a}_1 = -\frac{r_x(1)}{r_x(0)}$$

If we substitute the true AR(2) ratio into our forced AR(1) estimate, we get:

$$\tilde{a}_1 = \frac{a_1}{1 + a_2}$$

This proof shows exactly why underfitting is bad. Because we didn't give the model enough parameters to capture the true dynamics (it is missing a_2), the math forces the single parameter we *did* allow ($\tilde{a}_1$) to absorb the missing information. This permanently distorts the parameter and ruins the accuracy of the model.

3 Task 1: ECG Signal Enhancement

3.1 Time-Domain Analysis and Signal Morphology

The objective of this task is to analyze and enhance an electrocardiogram (ECG) signal sampled at $F_s = 500$ Hz that has been corrupted by powerline noise.

First, an inspection of the signal in the time domain (the whole signal and then using a zoomed-in 1-second window), as you can see clearly in Figure 1, reveals the standard physiological structures of a cardiac cycle. The most prominent feature is the sharp QRS complex, which represents rapid ventricular depolarization. Preceding it is the smaller P-wave (atrial depolarization), and following it is the T-wave (ventricular repolarization).

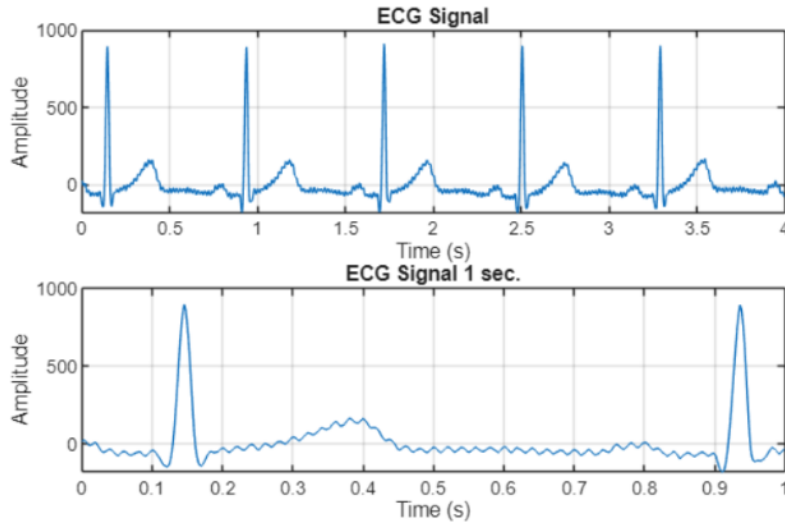


Figure 1: ECG Signal and ECG Signal 1 sec. zoom in

Superimposed over these physiological components is a continuous, high-frequency sinusoidal ripple as you can observe in the picture. This structure is a classic manifestation of powerline interference, caused by electromagnetic leakage from the local electrical grid into the recording instrumentation.

3.2 Spectral Analysis and Heart Rate Estimation

To quantify the interference and analyze the signal's harmonic structure, a zero-padded Fast Fourier Transform (FFT) was computed. The results can be observed in the following figure:

The magnitude spectrum shows a series of peaks rapidly decaying at lower frequencies, which correspond to the fundamental heartbeat and its integer multiples (harmonics). But, at exactly 50 Hz, there is an isolated spike in the spectrum. This confirms that the high-frequency ripple observed in the time domain is indeed the 50 Hz power grid interference.

Estimating the patient's heart rate requires identifying the fundamental frequency (f_0), which represents the repetition rate of the entire heartbeat cycle. The period (T_p) is the inverse of this frequency ($T_p = 1/f_0$). The implemented MATLAB script attempted to calculate this using two algorithmic methods:

1. **FFT Peak Distance:** The code identified peaks in the frequency spectrum and averaged the distance between them, yielding $f_0 \approx 1.96$ Hz (117 bpm).

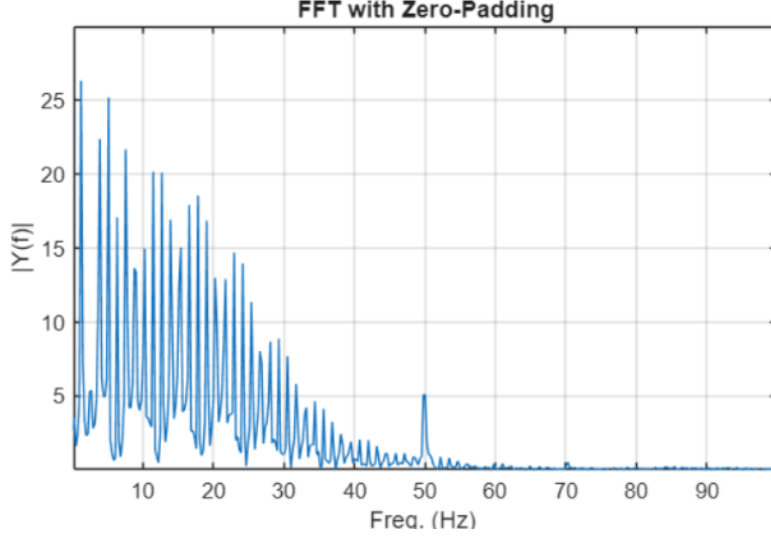


Figure 2: FFT with Zero-Padding

2. **Autocovariance (ACF):** By plotting the autocovariance (observed in Figure 3) and finding the lag time of the first major peak (ignoring the first 0.4 seconds to bypass lag zero), the algorithm estimated a period of 0.4 seconds, yielding $f_0 = 2.5$ Hz (150 bpm).

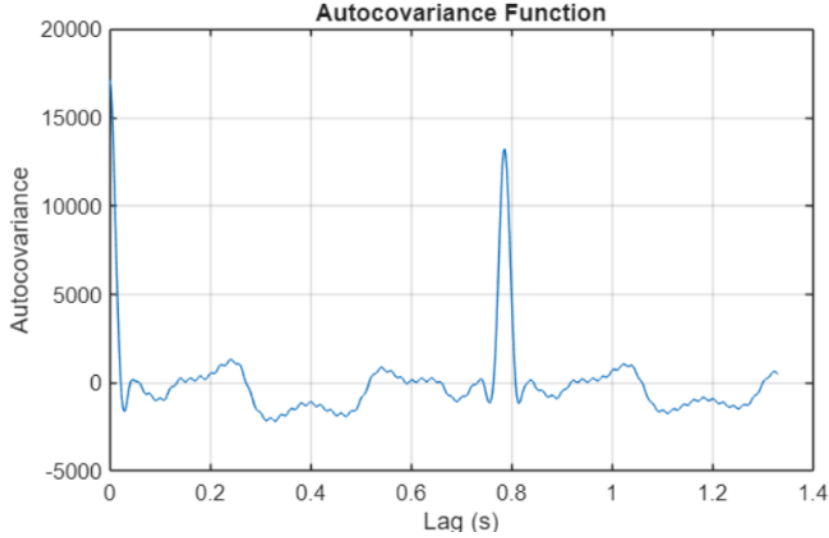


Figure 3: Autocovariance Function

As we can observe, both algorithmic outputs are artificially high compared to what we can see in the figures. Looking directly at the time-domain plot, the distance between the R-peaks (the period T_p) is approximately 0.8 seconds, which corresponds to a true fundamental frequency of $f_0 = 1.25$ Hz and a resting heart rate of approximately 75 bpm.

So, why did the FFT method fail? The simple `diff()` function captured false peaks caused by baseline wander and noise, messing with the mean harmonic distance.

And, why did the ACF method fail? The `ignore_samples` threshold was set to 0.4 seconds. The ECG signal contains a secondary morphology (the T-wave) that creates a minor correlation sub-peak exactly around 0.4 seconds. The algorithm caught this artifact instead of the true global maximum at 0.8 seconds.

3.3 IIR Filter Design and Implementation

To remove the 50 Hz interference, as you can see in the figure, a Butterworth Infinite Impulse Response (IIR) filter was designed.

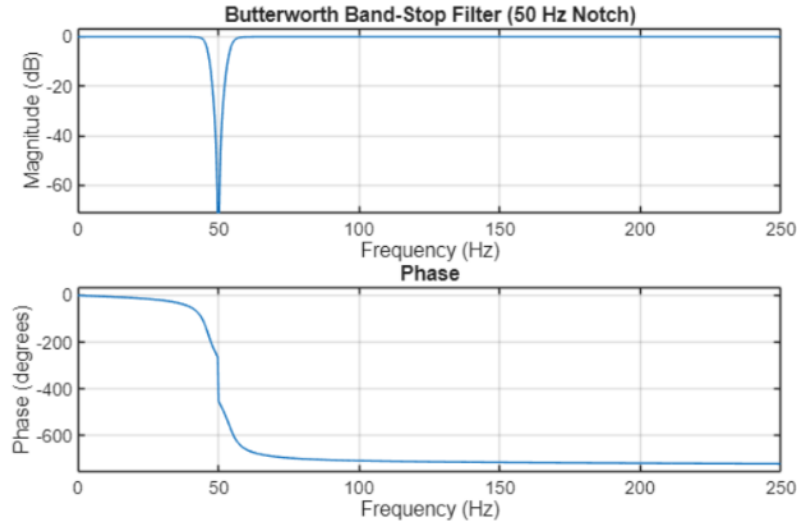


Figure 4: IIR Filter

A band-stop filter was selected. This selection was made because it was required to eliminate a single and narrow frequency band. A low-pass filter would destroy the high-frequency components critical for preserving the sharp morphology of the QRS complex. The passband edges were set at [45, 55] Hz and stopband edges at [49, 51] Hz. This creates a localized 2 Hz rejection band specifically tailored to the 50 Hz noise, ensuring minimal loss of adjacent physiological data.

Then, using `buttord`, the required filter order was calculated to be remarkably low ($n = 3$). The magnitude response successfully exhibits a steep 40 dB attenuation at 50 Hz. However, the phase response plot reveals severe non-linearity directly around the cutoff frequency.

3.4 Filter vs. `filtfilt`

When the ECG was processed using the standard `filter` command, the 50 Hz noise was successfully removed, but the resulting wave was visibly shifted to the right in time, which can be seen in Figure 5. This time delay and slight morphological distortion are direct consequences of the IIR filter's non-linear phase response: different frequency components are delayed by varying amounts (group delay).

To resolve this, the signal was processed using `filtfilt`. This function performs forward-backward filtering: processing the signal, reversing it in time, and filtering it again. This results in an output signal that fits perfectly with the R-peaks of the original signal, as can be observed in Figure 6.

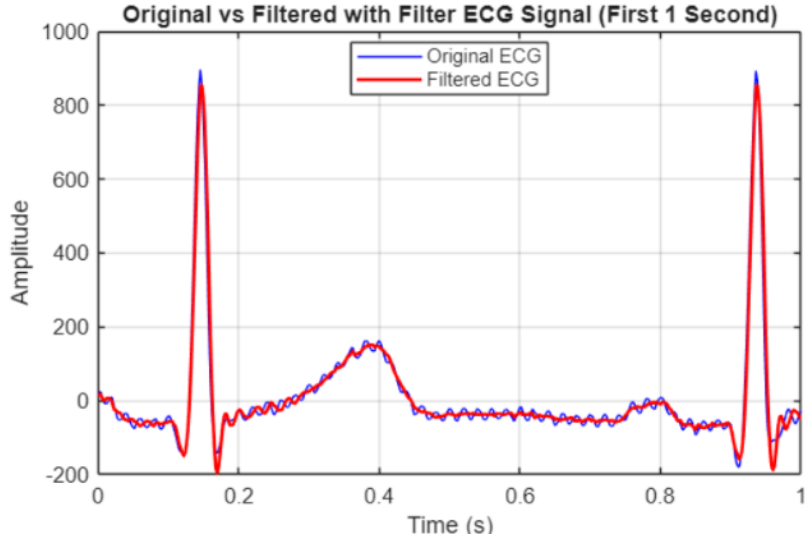


Figure 5: ECG Signal Filtered with Filter

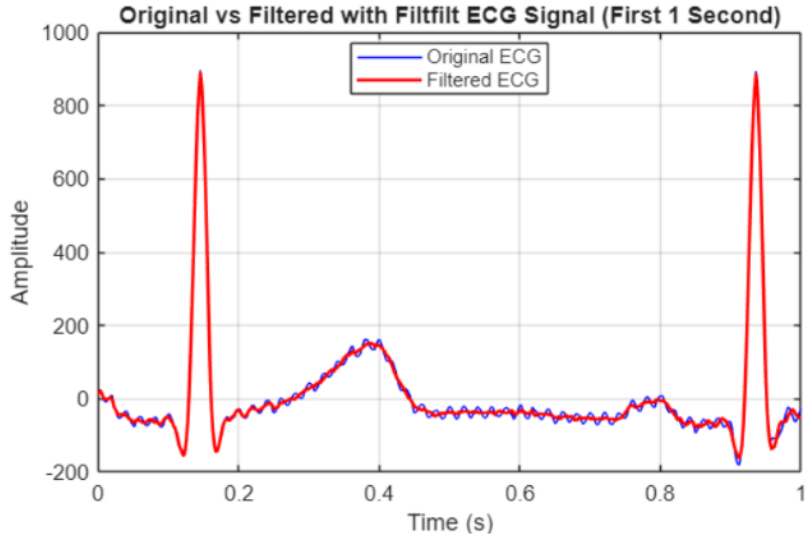


Figure 6: ECG Signal Filtered with Filtfilt

3.5 FIR Filter Comparison

For comparison, a Finite Impulse Response (FIR) filter was designed using the least-squares method (`firls`) with identical edge frequencies and a predefined order of $N = 400$.

Because FIR filters lack feedback loops (poles), they require a significantly larger number of coefficients (a high order) to achieve the steep transition bands of an IIR filter. The distinct advantage, however, is visible in the phase plot (Figure 7): a perfectly straight, diagonal line, indicating linear phase.

When applied using the `filter` command, the FIR filter perfectly preserved the wave's shape without distortion, as can be observed in Figure 8. However, it introduced a massive constant time delay. A linear phase FIR filter delays a signal by $N/2$ samples. With $N = 400$ and $F_s = 500$ Hz, this created a 200-sample lag, translating to a severe 0.4-second delay relative to the original signal.

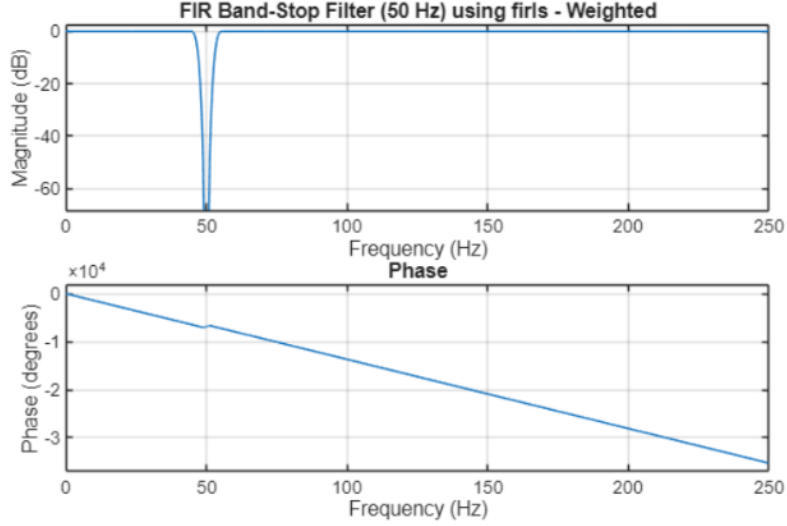


Figure 7: FIR Filter

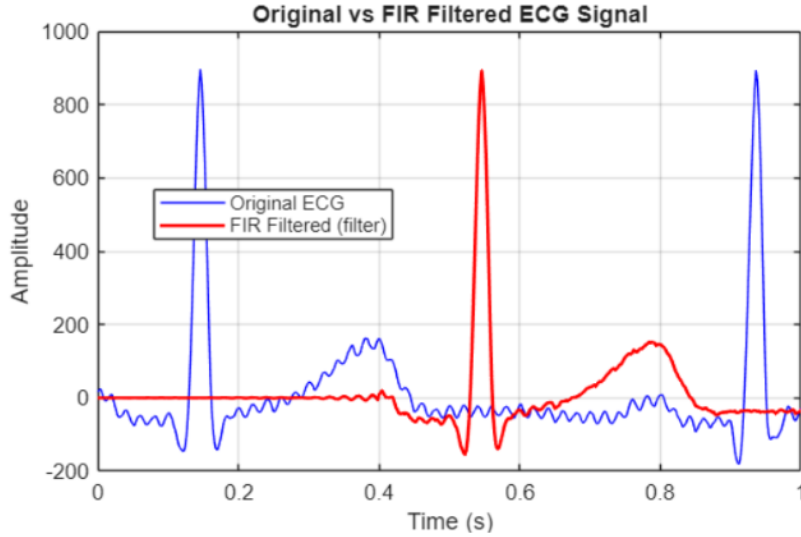


Figure 8: ECG Signal Filtered with FIR

3.6 Conclusion: Architectural Trade-offs

Both filter types successfully suppress powerline interference, but their architectural differences dictate their practical applications:

For **IIR filters**, high computational efficiency is achieved due to their low order, making them ideal for real-time applications (e.g., bedside ICU monitors or embedded clinical devices). However, they introduce phase distortion. In offline analysis, this is mostly solved using `filtfilt`.

On the other hand, **FIR filters** are inherently stable and guarantee linear phase (no shape distortion), but the high order required for sharp cutoffs introduces significant computational load and unacceptable real-time latency (0.4 s). They are best suited for applications where wave morphology must be preserved in real time, but only if the required filter order can be kept relatively low.

4 Task 2: AR-model estimation

4.1 Fitting AR(p)-models to the data

The objective of this task is to fit parametric models to data as a way of extracting information from noisy measurements.

First, we compute and plot the first 50 lags of the covariance of y using the function `covf`. We do this to observe the structure of the signal and determine whether we can extract some information or if it is white noise.

In our case, we can observe that the signal is not white noise. This is because if it were, the covariance at lag 0 would be significantly larger, while the covariance at most other lags would be close to 0. However, in our case, we can visualize a gradual oscillating trend with decaying covariance, leading us to conclude that there is some structure in the signal.

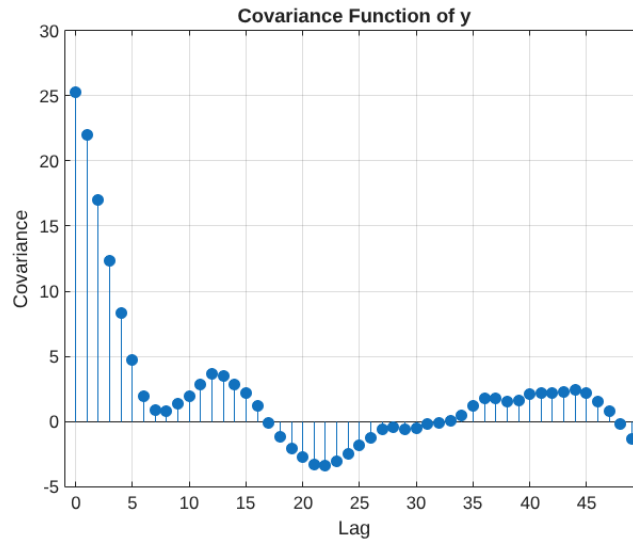


Figure 9: Covariance function of y

Next, we fit a model for each order from $p = 1$ to $p = 20$ and compute the prediction error variance, MDL, and AIC for each of them. Looking at the plot containing the aggregated metrics for each fitted model, we can observe that the curves vary in shape. The prediction error variance shows a sharp decrease at the beginning, between $p = 1$ and $p = 2$, followed by a gradual decreasing trend for the remaining orders. Meanwhile, both the AIC and MDL curves reach their minimum at $p = 2$ and then steadily increase for higher model orders.

These curves indicate that the optimal model order is $p = 2$, as confirmed by both the AIC and MDL criteria.

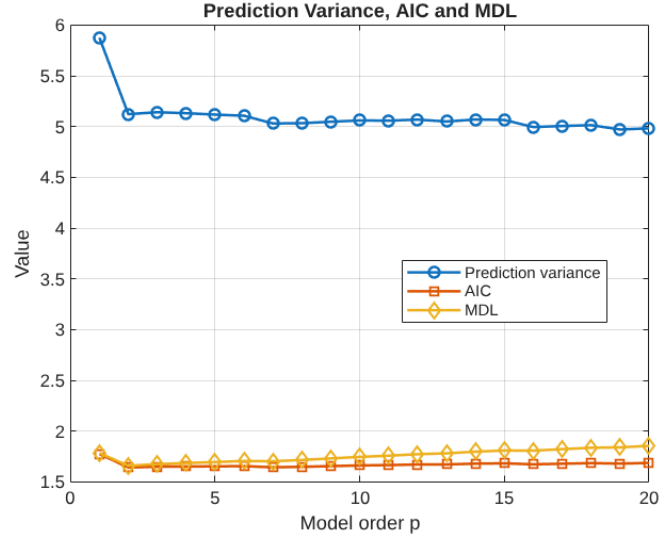


Figure 10: Metrics to evaluate the optimal order

4.2 Model with Optimal fit

The next task was to fit and analyze a model with the optimal order, $p = 2$. For this, we used the functions `ar_model0 = arx(iddata(y), p0)` and `present(ar_model0)`, provided by our instructor. These functions allow us to observe the estimated AR coefficients together with their confidence intervals so that we can evaluate the model.

The model coefficients were -1.20 ± 0.04195 and 0.365 ± 0.0421 , both statistically significant because their confidence intervals do not include 0. On the other hand, the resulting covariance plot (Figure 11) shows a large peak at lag 0, corresponding to the variance of the excitation noise, while the remaining lags fluctuate close to zero. This is the expected behavior for a correctly fitted AR model, as the excitation noise should behave similarly to white noise.

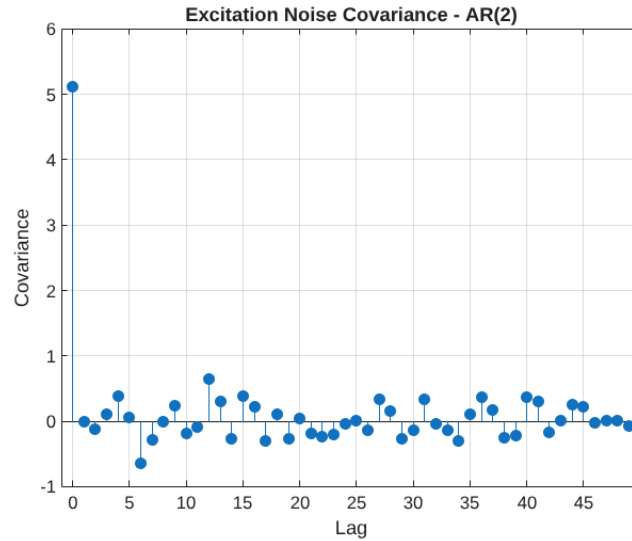


Figure 11: AR(2) Model Excitation Noise Covariance

4.3 Model with a "too large" fit and too "small fit"

To have a better comparison of the effect of correctly fitting a model, we also evaluated the $AR(p_0 + 1)$ and $AR(p_0 - 1)$ models.

For the case $AR(p_0 + 1)$, the first two coefficients of the model are -1.199 ± 0.04507 and 0.3634 ± 0.0684 , which are very similar to those of the optimal model. Adding one extra parameter did not modify the main dynamics of the model significantly. However, the third coefficient was 0.001226 ± 0.04527 . Its confidence interval includes zero, meaning it is not statistically significantly different from zero, so it does not add meaningful information about the signal.

The covariance function of the excitation noise has a behavior similar to that of the optimal model (Figure 12). It has a large peak at lag 0 and values close to zero for the remaining lags. The residual process is still approximately white noise.

In practical terms, this means that the model is more complex, but the added complexity does not improve the signal estimation. Therefore, between the two models, the $AR(p_0)$ model would be preferred.

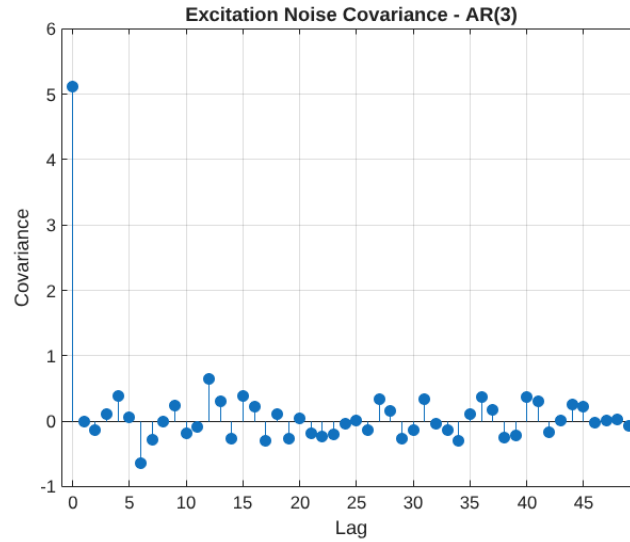


Figure 12: AR(3) Model Excitation Noise Covariance

For the case $AR(p_0 - 1)$, the coefficient of the model is 0.8816 ± 0.02172 , which differs from the first coefficient of the optimal model. Its value and confidence interval are still statistically significant because they do not include 0.

The excitation noise, however, presents a different behavior (Figure 13). There is still a large peak at lag 0; however, the residuals of the process show a structured pattern rather than simply fluctuating around 0 as white noise would.

In practical terms, this means that the $AR(p_0 - 1)$ model is too simple to describe the signal correctly. By using fewer parameters than the optimal model, part of the temporal correlation remains in the excitation noise, resulting in a poorer fit.

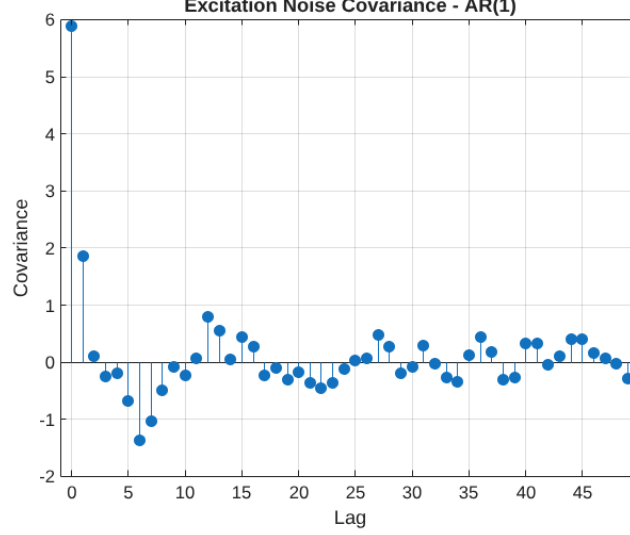


Figure 13: AR(1) Model Excitation Noise Covariance

4.4 Pre-lab task relation

In the preparation task, it was shown that when an AR(1) model is fitted to a signal generated by an AR(2) process, the estimated coefficient is distorted, following the equation

$$\tilde{a}_1 = \frac{a_1}{1 + a_2}.$$

Since the optimal model order in Task 2 was $p_0 = 2$, the AR($p_0 - 1$) model corresponds to an AR(1) model, so we can use its estimated coefficient to verify this theoretical result.

Using the coefficients of the optimal AR(2) model, $a_1 = -1.20$ and $a_2 = 0.365$, we obtain

$$\tilde{a}_1 = \frac{-1.20}{1 + 0.365} = -0.8791.$$

The estimated AR(1) coefficient from MATLAB was -0.8816 ± 0.02172 , which is very close to the theoretical value. This confirms the result from the preparation task. The AR(1) model is missing the second parameter, so the single coefficient absorbs part of the effect of the omitted AR(2) coefficient. As a result, the AR(1) model underfits the signal and does not represent the true dynamics of the AR(2) process.

4.5 Simulation Study

To evaluate how reliable the model order selection criteria are, a simulation study was performed using the optimal AR(p_0) model.

We generated 100 signals with 500 samples each, and for each signal we fitted AR models of orders 1 to 20. Then, using AIC and MDL, we selected the optimal model order for each simulation and plotted the results in a histogram shown in Figure 14.

In both cases, the most frequently selected order was $p = 2$, but the distribution of each method was slightly different. The AIC histogram shows a wider spread, with several simulations

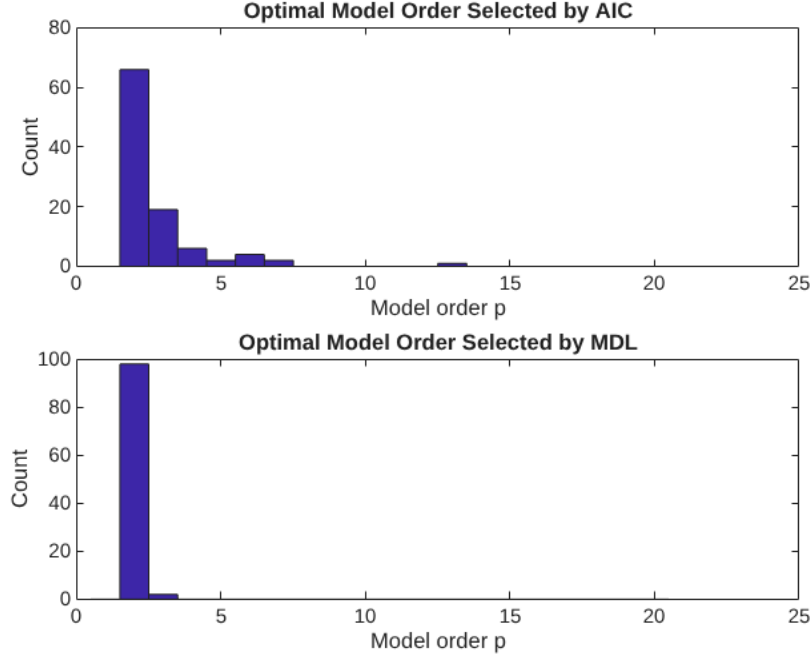


Figure 14: Simulation histograms

selecting higher model orders, such as AR(3) and AR(4). In contrast, the MDL histogram is more concentrated around $p = 2$.

Thanks to the results of the simulation, we can conclude that AIC has a tendency to choose more complex models, whereas MDL applies a stronger penalty, making it less likely to overfit the data.

5 Task 3: EEG-signal classification for brain-computer interface

The main objective of this task is to investigate whether EEG signals recorded during motor imagery can be effectively utilized for a Brain-Computer Interface (BCI), specifically, determining if it is possible to mathematically distinguish between the user’s intent to move their “left hand” versus their “right foot”. To achieve this, we shift from basic signal reconstruction to feature extraction. In this case, the estimated Autoregressive (AR) coefficients (a_k) serve as a representative feature vector of the user’s brain state, capturing the underlying spectral profile of the motor cortex during imagined movement.

As shown in Figure 15, the raw EEG data in the time domain reveals highly complex, noisy, and stochastic signals. Because it is exceedingly difficult to visually distinguish between the “left hand” and “right foot” states using just the raw time-series morphology, we must rely on parametric modeling to extract distinguishable features.

5.1 Individual Signal Analysis (Left Hand vs. Right Foot)

Before determining a global model order, we analyzed the first 2-second recording from both the left hand and right foot datasets. For each recording, AR(p) models were fitted for $p = 1, 2, \dots, 20$.

Figure 16 shows the prediction error variance, AIC, and MDL for the left hand recording.

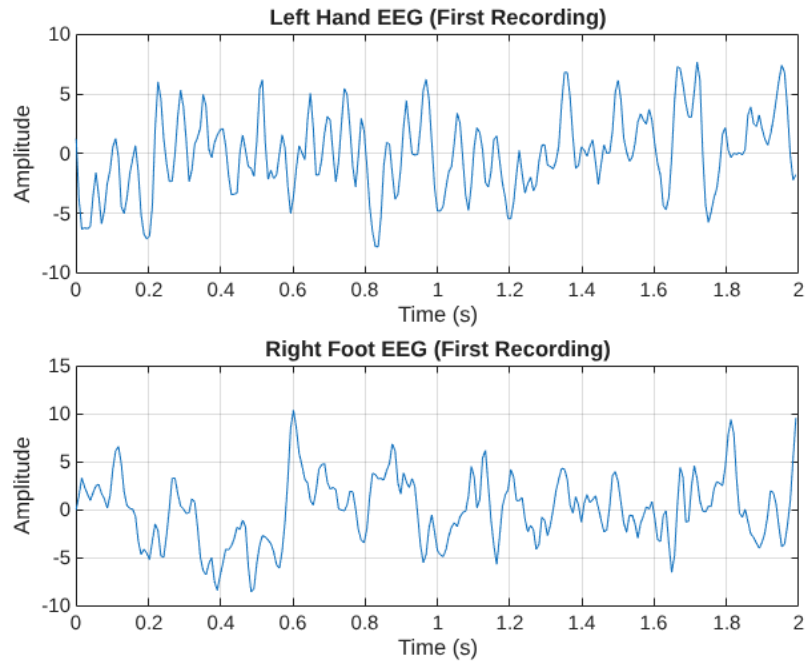


Figure 15: Time-domain representation of the first 2-second EEG recording for left hand (top) and right foot (bottom) motor imagery.

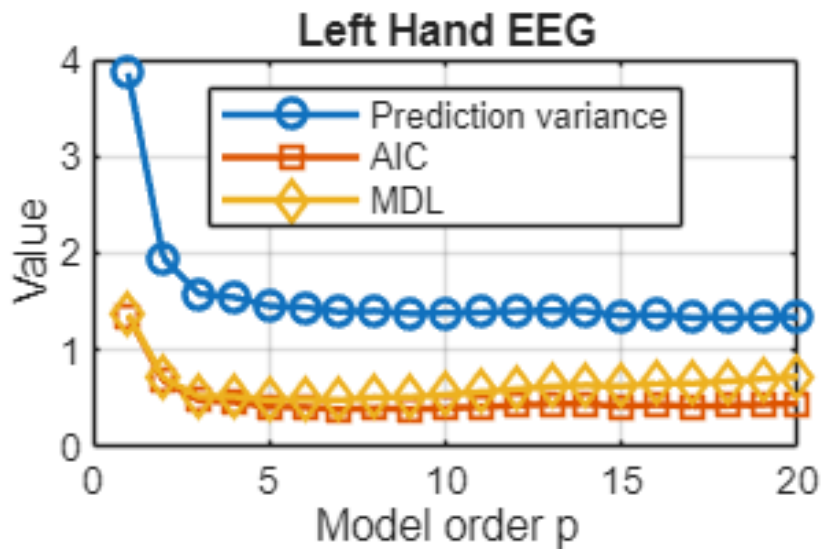


Figure 16: Prediction error variance, AIC, and MDL as a function of model order p for the first Left Hand EEG recording.

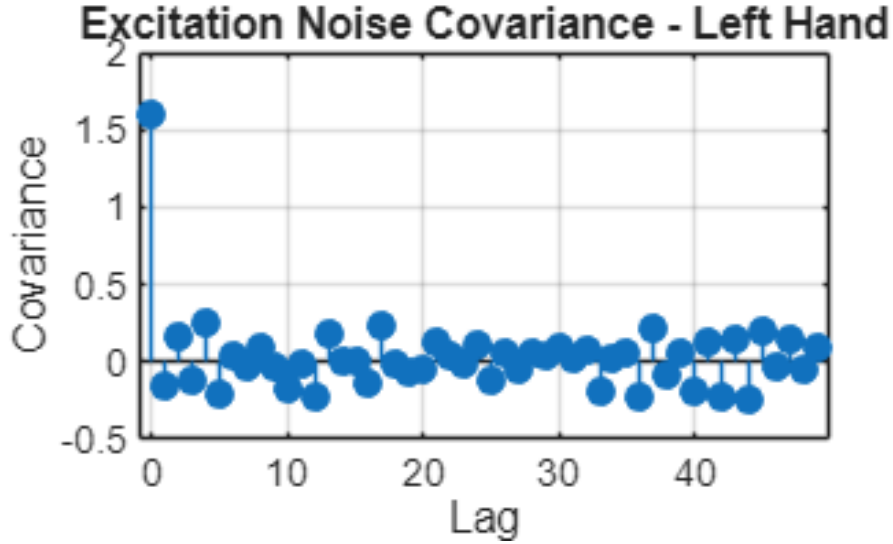


Figure 17: Excitation noise covariance for the optimal AR(5) model of the first Left Hand EEG recording.

The prediction variance naturally decreases as model complexity increases. However, AIC and MDL penalize this complexity to prevent overfitting. For this specific trial, MDL identified the optimal model order as $p = 5$. When filtering the signal to estimate the excitation noise of this AR(5) model (Figure 17), the covariance exhibits the expected behavior of white noise: a strong peak at lag 0 and near-zero values at all other lags, confirming the model accurately captured the signal's structure.

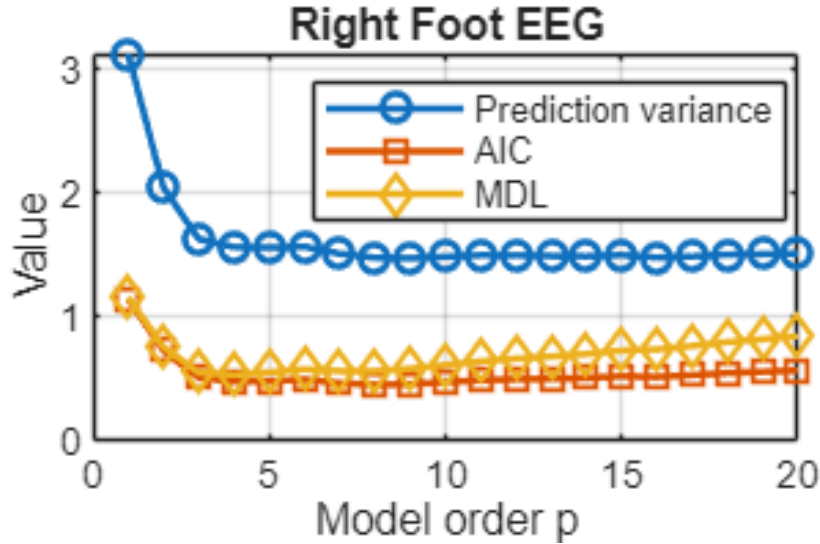


Figure 18: Prediction error variance, AIC, and MDL as a function of model order p for the first Right Foot EEG recording.

Applying the same procedure to the first right foot recording (Figure 18), the optimal order according to MDL was found to be $p = 4$. Its corresponding excitation noise covariance (Figure 19) similarly validates the model by displaying characteristic white noise properties. The variance in optimal orders between individual recordings underscores the necessity of establishing a single, common model order for classification purposes.

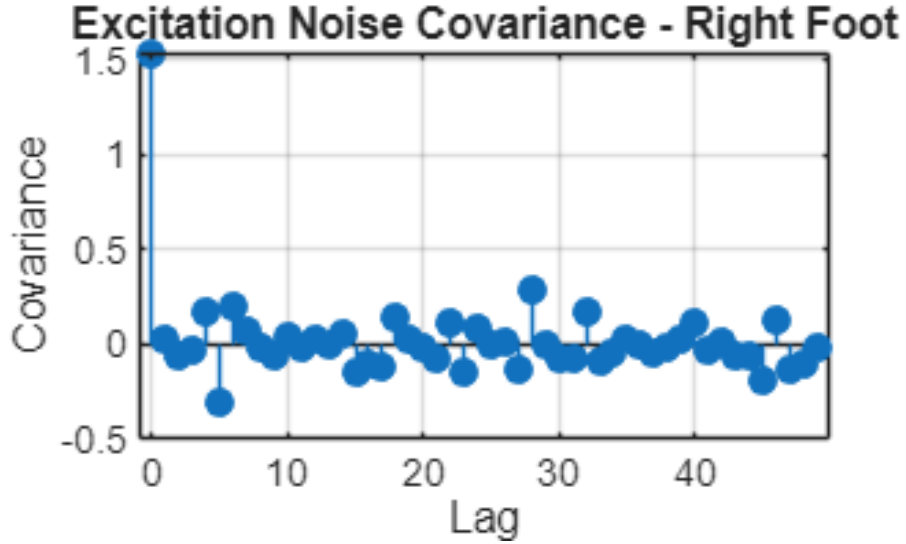


Figure 19: Excitation noise covariance for the optimal AR(4) model of the first Right Foot EEG recording.

5.2 Selecting a Common Model Order (p_0)

To effectively train a classifier to distinguish between “left hand” and “right foot” imagery, we have to compare feature vectors of the exact same length. Comparing an AR(2) model to an AR(5) model would be comparing fundamentally different mathematical spaces. Therefore, a single global model order (p_0) must be chosen for all recordings.

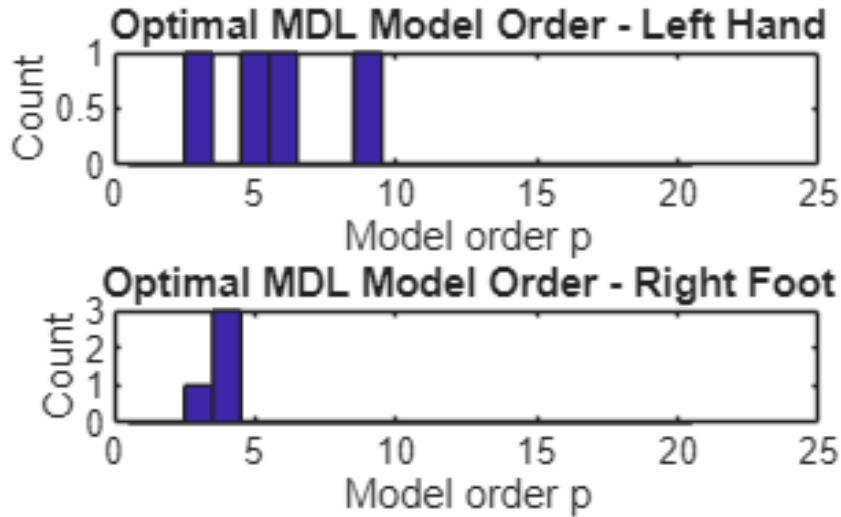


Figure 20: Histograms of the optimal MDL model orders across all Left Hand (top) and Right Foot (bottom) recordings.

To determine this, the optimal Minimum Description Length (MDL) order was calculated for every 2-second trial across both datasets. The distribution of these optimal orders is shown in Figure 20. The optimal common model order p_0 was selected by taking the mode (the most frequent value) of all the optimal orders. The algorithm determined that $p_0 = 4$ is the globally optimal choice.

Selecting the mode ensures that we are using the model order that is statistically optimal

for the vast majority of the data. An AR(4) model provides enough parameters to capture the low-frequency oscillatory dynamics typical of motor imagery EEGs, while strictly preventing the overfitting that would occur if we catered to the noisy outliers.

5.3 Distinguishability of AR Coefficients

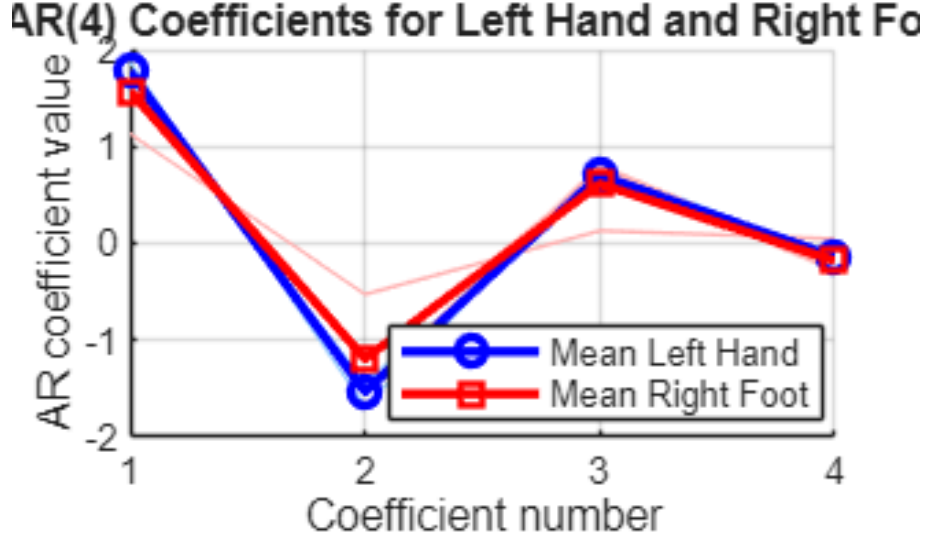


Figure 21: AR(4) coefficients for individual recordings and class means for Left Hand and Right Foot datasets.

With the global order set to $p_0 = 4$, AR(4) models were fitted to all individual recordings across both datasets. The estimated AR coefficients (a_1 through a_4) for each recording are visualized in Figure 21. The faint lines represent the coefficient variations across individual trials, while the bold lines with markers represent the mean coefficient values for each class.

- **Class Separation:** Observing the coefficient space, it is indeed possible to distinguish between the “left hand” and “right foot” signals. While the individual trial lines (light blue and light red) shows significant variance and overlap, a standard characteristic of noisy, non-stationary EEG data, the robust mean vectors (solid blue and red lines) clearly diverge.
- **Promising Features:** The lower-order coefficients (specifically a_1 and a_2) generally offer the most promising separation. Because the lower-order parameters capture the dominant, broad spectral peaks of the signal, they heavily reflect the primary physiological shifts in the brain. The higher-order coefficients (a_3 and a_4) tend to model finer, more nuanced spectral details, resulting in means that converge closer together and offer less discriminative power.

5.4 Application in a BCI System

In a practical Brain-Computer Interface application, this AR modeling pipeline forms the core “Feature Extraction” stage. The system would operate as follows:

1. **Continuous Acquisition:** The BCI hardware records the user’s EEG continuously, buffering the data into short, overlapping time windows.
2. **Real-Time Feature Extraction:** For every incoming window, the system rapidly computes the Yule-Walker equations to estimate the AR(4) coefficients (a_1, a_2, a_3, a_4). These four numbers form the input “feature vector.”

3. **Machine Learning Classification:** This vector is fed into a pre-trained machine learning classifier (such as a Linear Discriminant Analysis (LDA) or Support Vector Machine (SVM) algorithm). The classifier evaluates the coefficients' location in the feature space—relying heavily on the distinct separation found in a_1 and a_2 —and assigns a probability to the user's intent.
4. **Control Output:** The classification result is immediately translated into a machine command, allowing the user to steer a motorized wheelchair, or move a computer cursor, solely through imagined movement.